

1. FileWriter – Write to a file (Character Stream)

```
FileWriter fw = new FileWriter("hds.txt"); //file gets created if not exists
fw.write("Hello from FileWriter!\n");
fw.write(100);
fw.close(); //or fw.flush();
```

2. FileReader – Read from a file (Character Stream)

```
FileReader fr = new FileReader("hds.txt"); //file must exists
int data = fr.read();
while (data != -1) {
    System.out.print((char) data);
    data = fr.read();
}
```

3. BufferedWriter – Efficient character writing using `newLine()`:

```
BufferedWriter bw = new BufferedWriter(new FileWriter("buffered.txt"));
bw.write("Hello from BufferedWriter!");
bw.newLine();
bw.write("Another line.");
bw.close(); // or bw.flush();
```

4. BufferedReader – Efficient character reading using `readLine()`:

```
BufferedReader br = new BufferedReader(new FileReader("buffered.txt"));
String line = br.readLine();
while (line != null) {
    System.out.println(line);
    line = br.readLine();
}
```

5,6. FileOutputStream – Write bytes to a file (Byte Stream) and FileInputStream – Read bytes from a file (Byte Stream)

```
FileOutputStream fos = new FileOutputStream("output.jpg");
FileInputStream fis = new FileInputStream("input.png");
```

Use `import java.io.*;` and handle `IOException` in every case

```
int i = fis.read();
while (i != -1) {
    fos.write(i);
    i = fis.read();
}
fos.close();
```

```
Write and Read Character Stream using FOS - FIS
FileOutputStream fos = new FileOutputStream("output.txt");
String text = "Data from FileOutputStream";
fos.write(text.getBytes()); fos.write(97); fos.write('b');
fos.close();

FileInputStream fis = new FileInputStream("example.txt");
int data;
while ((data = fis.read()) != -1) {
    System.out.print((char) data);
}
```

7. InputStreamReader with `System.in` – take user input and convert to character stream

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
```

```
String input = br.readLine(); //will ask for user input in terminal
System.out.println("You typed: " + input);
FileWriter fw = new FileWriter("buffered.txt");
fw.write(input);
fw.close();
```

8. InputStreamReader – read data and convert to character stream

```
FileInputStream fis = new FileInputStream("bytefile.txt");
InputStreamReader isr = new InputStreamReader(fis);
int data = isr.read();
while (data != -1) {
    System.out.print((char) data);
    data = isr.read();
}
```

File Class Constructors:

- `File(String pathname);`
- `File(String parent, String child);`
- `File(File parent, String child);`

Methods of the File Class

```
import java.io.*;
import java.util.*;

public class FileClassExample {
    public static void main(String[] args) throws IOException {
        File file = new File("example.txt");
        if (file.createNewFile())
            System.out.println("File created.");
        else
            System.out.println("File already exists.");

        File directory = new File("D://src/newDir"); // or new File("D://src", "newDir");
        if (directory.mkdir())
            System.out.println("Directory created.");
        else
            System.out.println("Directory already exists.");

        System.out.println("File exists: " + file.exists());           // boolean true if exists
        System.out.println("Is a File: " + file.isFile());           // boolean true if it is file
        System.out.println("Is a Directory: " + file.isDirectory()); // boolean if it is folder
        System.out.println("File Name: " + file.getName());         // returns String - file name
        System.out.println("File Path: " + file.getPath());         // returns String - file path
        System.out.println("Absolute Path: " + file.getAbsolutePath()); // returns String - file full path - D:/...
        System.out.println("Parent Directory: " + file.getParent()); // returns String - parent dir name
        System.out.println("File Size (bytes): " + file.length());   // returns file size in bytes
        System.out.println("Readable: " + file.canRead());          // boolean true if readable
        System.out.println("Writable: " + file.canWrite());          // boolean true if writable
        System.out.println("Executable: " + file.canExecute());      // boolean true if executable

        File anotherFile = new File("example.txt");
        System.out.println("Files Equal?" + file.equals(anotherFile)); // boolean true if both objects refer
        same file

        System.out.println("Files in 'src' directory:"); // 'src' folder must exist in your project
        File currentDir = new File("src");

        String[] files = currentDir.list(); // list out all file/folder names in string
        if (files != null) {
            for (String f : files) {
                System.out.println(f); } }

        File[] fileList = currentDir.listFiles(); // list out all file/folder paths in File Object format
        if (fileList != null) {
            for (File f : fileList) {
                System.out.println((f.isDirectory() ? "Dir " : "File ") + f.getName()); } } }
```

RandomAccessFile Class Constructors:

- `RandomAccessFile(File file, String mode):`
- `RandomAccessFile(String filename, String mode):`

The mode can be `"r"` for read-only access, or `"rw"` for read-write access.

Methods of the RandomAccessFile Class

`RandomAccessFile` allows you to read and write structured data at any position within a file.

1. `void close():` Closes the file stream and releases any system resources.
2. `int readInt():` Reads a 4-byte integer from the file.
3. `String readUTF():` Reads a string encoded in UTF-8 format from the file.
4. `void seek(long pos):` Sets the file pointer to a specific position (offset) in the file. (seek(0); pointer goes back to beginning, seek(raf.length()); goes to end)
5. `void writeDouble(double v):` Writes an 8-byte double to the file.
6. `void writeFloat(float v):` Writes a 4-byte float to the file.
7. `void write(int b):` Writes a single byte to the file.
8. `void write(byte[] b, int offset, int len):` Writes bytes initialising from offset position upto b.length from the buffer.
9. `void writeUTF(String str):` writes a string to file using modified UTF-8 encoding(machine-independent)
10. `int read():` Reads single byte from file. Returns -1 if the end of file is reached.
11. `long length():` Returns the length of the file in bytes.
12. `long getFilePointer():` Returns the current position of the file pointer.

```
RandomAccessFile raf = new RandomAccessFile("data.txt", "rw");
raf.writeInt(10);           // Writes 4 bytes (int: 10)
raf.writeUTF("Hi");         // Writes length + "Hi"
raf.writeDouble(5.5);       // Writes 8 bytes (double)
raf.writeFloat(2.5f);       // Writes 4 bytes (float)
raf.write(97);              // Writes 1 byte (ASCII of 'a')
byte[] data = {10, 20, 30, 40, 50};
raf.write(data, 1, 3);       // Write 3 bytes starting from index 1 (that's 20, 30, 40)
raf.seek(0);                // Moves pointer to beginning, for end: raf.seek(raf.length());

System.out.println("Int: " + raf.readInt());           // Int: 10
System.out.println("UTF: " + raf.readUTF());           // UTF: Hi
System.out.println("Double: " + raf.readDouble());     // Double: 5.5
System.out.println("Float: " + raf.readFloat());       // Float: 2.5
System.out.println("Byte: " + (char) raf.read());      // Byte: a
System.out.println("Pointer: " + raf.getFilePointer()); // Pointer: (position after reading above all)
System.out.println("Length: " + raf.length());         // Length: (total bytes written)
raf.close();     // Closes the file
```